The following code will make sure the database structure is set up and empty, ready for your specs to add data to it:

```
Sequel.extension :migration
Sequel::Migrator.run(DB, 'db/migrations')
DB[:expenses].truncate
```

First, we run all the migration files in to make sure that all the database tables exist with their current schema. Then, we remove any leftover test data from the table using the truncate method. That way, each run of the spec suite starts with a clean database—no matter what happened before.

The only problem is that it's not obvious where to put these lines. Up to this point, you've tended to keep setup routines like this in one of two places:

- The top of a single spec file
- The global spec_helper.rb file

Database migrations sit somewhere in between these two extremes. We want to load them for any spec that touches the database, but not for unit specs—those need to stay snappy, even as our database migrations grow over the lifetime of the application.

The RSpec convention for this kind of "partially shared" code is to put it in a folder called spec/support; then we can load it from whatever spec files need it. Create a new file called spec/support/db.rb with the following content:

06-integration-specs/03/expense_tracker/spec/support/db.rb

```
RSpec.configure do |c|
  c.before(:suite) do
    Sequel.extension :migration
    Sequel::Migrator.run(DB, 'db/migrations')
    DB[:expenses].truncate
  end
end
```

This snippet defines a *suite-level hook*. We first encountered before hooks in *Hooks*, on page 9. A typical hook will run before each example. This one will run just once: after all the specs have been *loaded*, but before the first one actually *runs*. That's what before(:suite) hooks are for.

Bootstrap Your Environment for Easy Testing



Your spec suite should set up the test database for you, rather than requiring you to run a separate setup task. People testing your code (including you!) can easily forget to run the extra step, or might not even know they need to.